

Observability, Reliability, and Incident Response

Observability helps answer questions about system behavior; reliability connects that behavior to user needs. Metrics, logs, and traces are complementary evidence. A useful incident process prioritizes mitigation, clear coordination, and later learning without substituting blame or dashboards for a tested recovery plan.

For: Engineers who want to move beyond adding logs and learn how to detect user impact, investigate a failure, mitigate safely, and improve the system after an incident.

Suggested pace: 3 weeks with controlled failure drills | A small service or project

What you will be able to do

- Define service-level indicators and objectives tied to user-visible behavior.
- Choose metrics, logs, and traces that discriminate likely failure causes.
- Design actionable alerts and bounded incident-response steps.
- Run a blameless follow-up that produces verified engineering improvements.

Build the right mental model

Measure the user contract

Availability should describe successful useful work, not merely an open port. Latency objectives need a population and percentile or threshold definition. Distinguish planned exclusions and error categories explicitly so an impressive number does not hide user failures.

Signals answer different questions

Metrics show trends and aggregates, logs provide event detail, and traces connect work across services. Correlate them using safe identifiers. Avoid unbounded high-cardinality labels or sensitive payloads that make the telemetry system expensive and risky.

Mitigation and explanation have different timelines

During an incident, restore service using the safest supported action while preserving evidence. Root-cause analysis can follow. A rollback may be appropriate before every causal detail is known, but it should be validated and communicated clearly.

Study with evidence.

Practice only on a local or approved test service. Write a small runbook, inject a reversible failure, and have a peer follow the instructions. Evaluate whether the alert and evidence lead to the right decision, not just whether a dashboard changes color.

Week 1: Define useful signals

Choose one user journey and define success, latency, and acceptable errors. Instrument request count, duration, failure, dependency time, and saturation. Add structured identifiers while excluding credentials and private data.

Deliverable and check: A service-level definition and a dashboard that reflects the actual journey rather than only host health.

Week 2, first half: Alerts and diagnosis

Create an alert for sustained user impact with an owner and response action. Use traces or correlated logs to separate application work from dependency delay. Review label cardinality and retention cost.

Deliverable and check: An actionable alert and a diagnostic decision tree supported by available evidence.

Week 2, second half: Failure drill

Introduce a slow local dependency, exhausted connection pool, or bad configuration. Record detection time, hypotheses, mitigation, and validation. Keep the failure reversible and avoid broad destructive recovery commands.

Deliverable and check: A drill timeline showing how the evidence led to a safe mitigation.

Week 3: Learn and verify

Write a blameless review covering impact, contributing factors, what worked, and specific follow-ups. Assign owners and validation criteria. Add a regression test or runbook improvement and repeat the relevant scenario.

Deliverable and check: A completed improvement with proof it addresses the observed failure, not an unowned list of recommendations.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Diagnose rising latency with stable CPU

An API's p99 latency rises and errors appear, but application CPU stays moderate. Database connection-pool wait time grows, and a recent release added a transaction that performs a slow external call while holding a connection.

A worked approach

1. Confirm user impact and compare the release timeline with request traces. Separate pool wait, database execution, and external-call duration. Moderate CPU does not rule out saturation of a different resource.
2. Mitigate using a validated rollback or a safe feature disable while preserving evidence. Confirm the old version remains compatible with the schema and observe whether pool wait and user errors recover.
3. Refactor the workflow so the slow external operation does not hold the database resource unnecessarily, preserving required correctness through a durable state machine or outbox as appropriate. Add a regression test with an intentionally slow provider.

Expected result

The incident is explained through a measured resource bottleneck and a release-specific behavior. The follow-up reduces connection hold time without weakening transactional or business guarantees.

Verify the result

- Did the mitigation improve user-visible latency and errors?
- Was the connection wait distinguished from query execution?
- Does the regression test fail on the original resource-holding behavior?

Extend the exercise

Estimate how an increased pool size could temporarily mask the defect while increasing database pressure. Explain why capacity tuning without root-cause reasoning may move the outage rather than solve it.

Make the learning stick.

Are more logs always better?

No. Excess volume, sensitive data, inconsistent structure, and high-cardinality fields can make investigation harder and increase cost. Collect evidence tied to meaningful questions with a deliberate retention and access policy.

What makes an alert actionable?

It indicates a condition requiring timely human action, identifies the affected service or journey, has an owner, and links to a useful response path. Alerts that repeatedly require no action train responders to ignore them.

Does a blameless review mean no accountability?

No. It focuses on system conditions and decisions rather than personal blame while still assigning concrete follow-up ownership. Accountability is strengthened when actions have verification criteria and are completed.

Avoid these traps

- Alerting on every unusual metric without a response action.
- Logging credentials or private payloads during diagnosis.
- Closing an incident without validating mitigation and follow-up work.

Ready for the next step?

- Indicators reflect a defined user journey.
- Telemetry can distinguish plausible failure causes.
- A safe mitigation and rollback path are documented.
- Post-incident improvements have owners and verification.

Reliable next reads

- [Google SRE book](#)
- [Google SRE workbook](#)
- [OpenTelemetry observability primer](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.